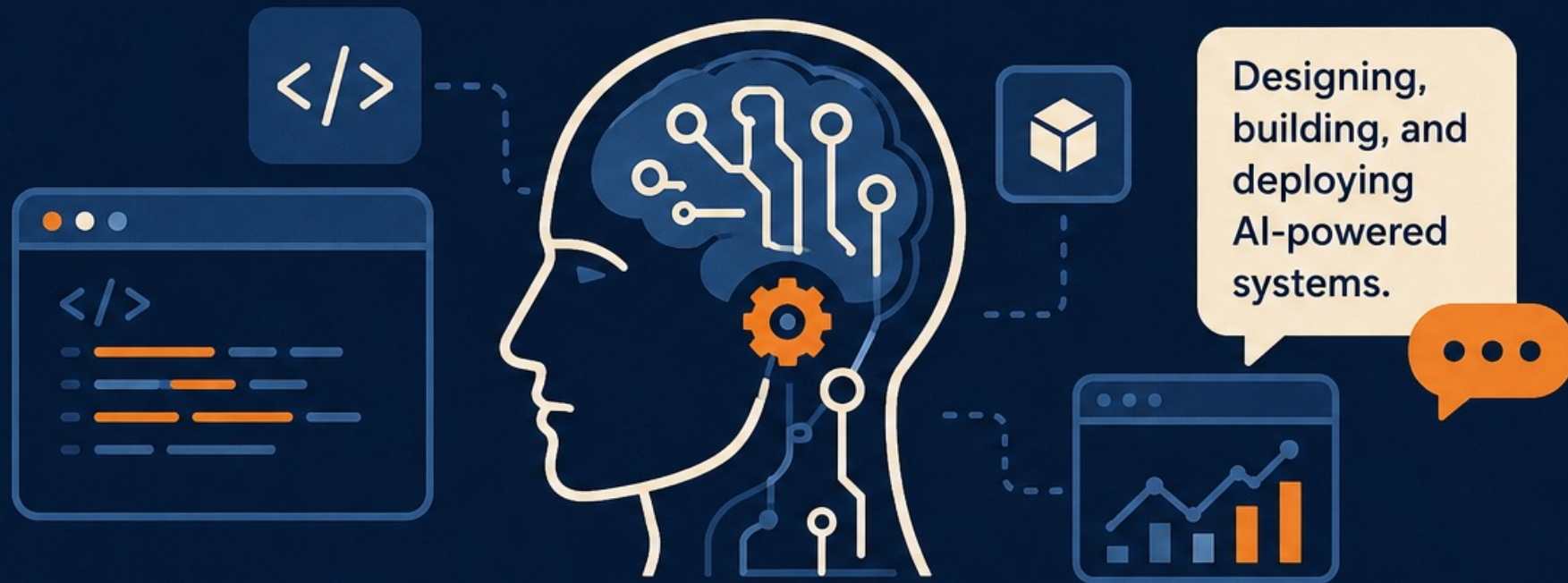


class starts
@ 9:05 pm

AI ENGINEERING



SYSTEM DESIGN



ML & AI
FUNDAMENTALS



DATA
ENGINEERING



MODEL
DEPLOYMENT



MLOPS



ETHICS &
RESPONSIBILITY

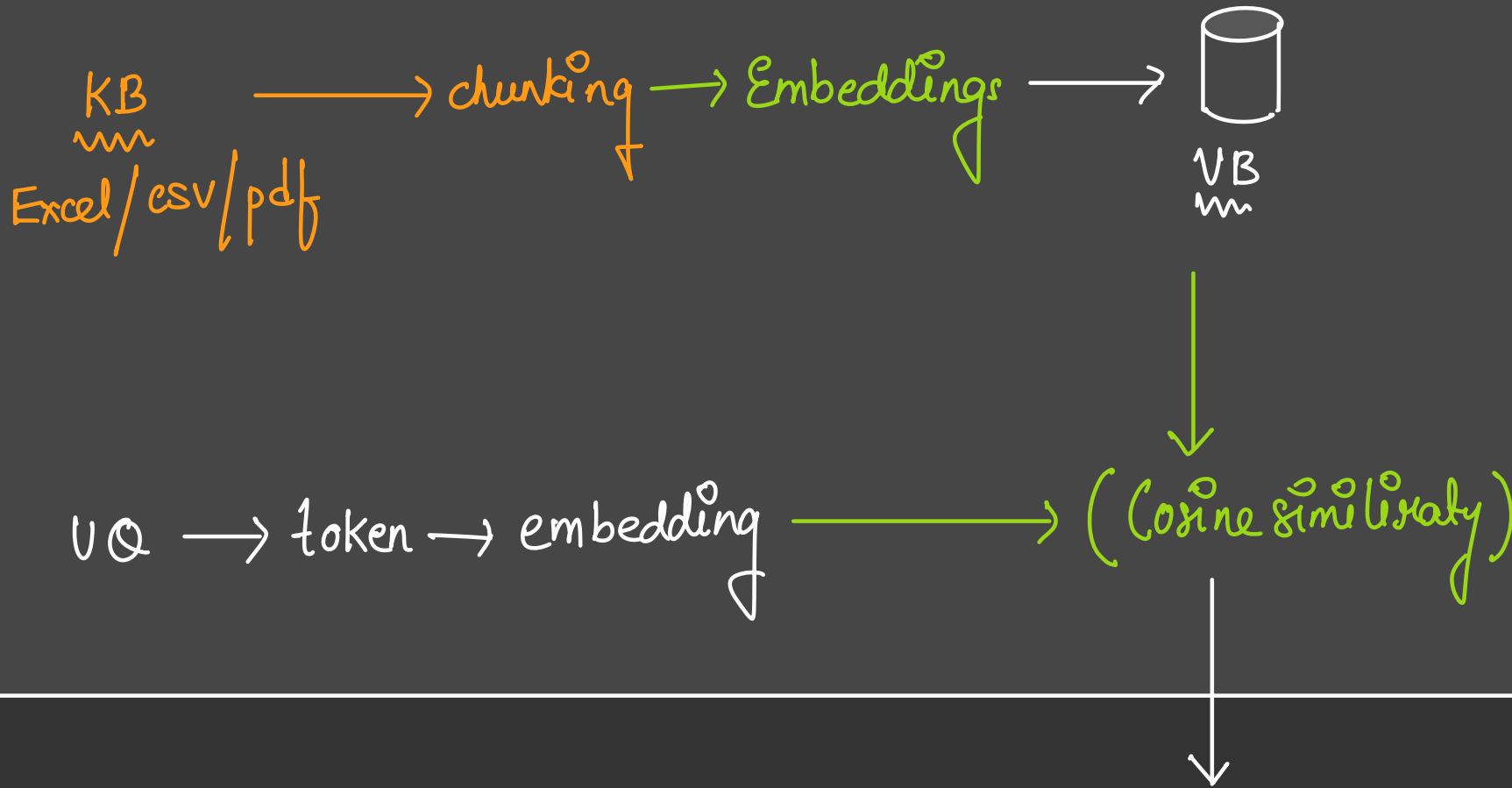
Lecture: Hybrid Search and Advanced Retrieval

Agenda

RAG

- 1) Hybrid Retrieval
- 2) Advanced Retrieval
 - Reformulation query
 - Confidence of a retrieval
 - Self correcting RAG

R A G



IN241

1 Hybrid Search

Combining the Best of BM25 and Dense Search

Hybrid search combines lexical matching (**BM25**) and semantic understanding (**Dense Search**) to deliver better retrieval quality.



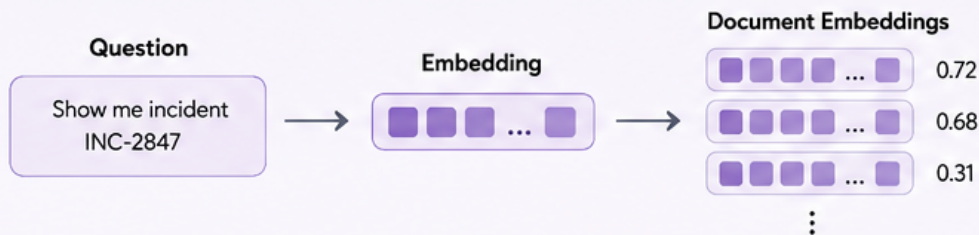
Goal

Retrieve documents that match the **exact words** as well as the **meaning** of the query.



Dense Search (Semantic)

Uses embeddings to capture meaning and context.



Understands meaning (e.g., “expired certificate” ≈ “SSL issue”)

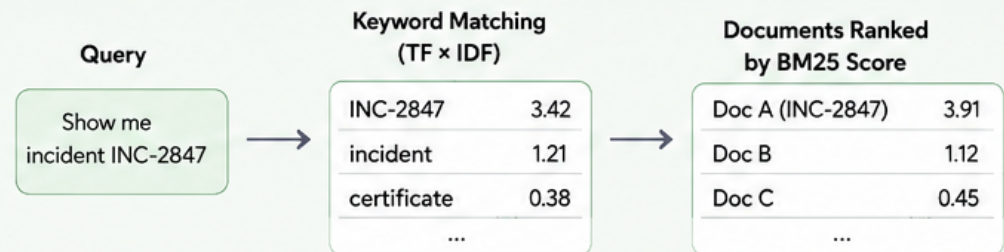


Struggles with exact identifiers (e.g., “INC-2847”)



BM25 (Lexical Search)

Uses keyword matching and term importance (TF × IDF).



Excellent for exact words, IDs, error codes, numbers

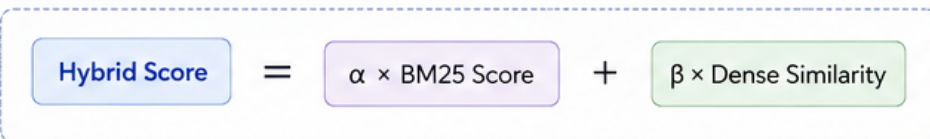


Poor at synonyms and paraphrases



Hybrid Search (Combination)

Combines lexical precision with semantic understanding.



Example: BM25 = 0.91 Dense = 0.72 → **Hybrid = 0.82**

Why Hybrid Works Best



Captures exact identifiers

(e.g., INC-2847, error 503, API-KEY)



Understands meaning

(e.g., prompt attack ≈ prompt injection)



Higher recall + better precision

Delivers more relevant results overall

When to Use

- ✓ Enterprise search
- ✓ Document retrieval
- ✓ Knowledge bases
- ✓ RAG systems



Key Takeaway

Hybrid Search leverages the strengths of BM25 and Dense Search, ensuring we don't miss critical documents due to synonym mismatch or lack of exact keyword match.

BM25
~~~~~

"Hello my name is ABC, looking for Incident INC 4821"

TFIDF  
~~~~~  
↓

TFIDF
~~~~~  
↑

→ TFIDF  
~~~~~

→ Term frequency & Inverse Document Frequency

Calculating IDF

Q $\rightarrow$ 2 terms
Machine

IDF

$$df(\text{machine}) = 2$$

$$IDF(q) = \ln \left(\frac{N - df(q) + 0.5}{df(q) + 0.5} + 1 \right)$$

$$\Rightarrow \ln \left(\frac{3 - 2 + 0.5}{2 + 0.5} + 1 \right)$$

$$\rightarrow \underline{0.47}$$

✓ Learning

$$IDF(\text{learning}) = 0.134$$

BM25 for D_1

$$|D_1| = 4$$

Term freq (machine, D_1) = 1
Term freq (learning, D_1) = 1

$$b = 0.75$$

$$\text{avg doc length} = \frac{4+4+4}{3} = 4$$

$$\left(1 - b + \frac{b |D|}{\text{avgdl}} \right) \rightarrow \text{denom}$$

$$\rightarrow \frac{1 - 0.75 + \frac{0.75(4)}{4}}{4} = \frac{1}{4} \checkmark$$

Contribution of machine

$$\left\{ \underbrace{0.47}_{\text{IDF}} \times \frac{\underbrace{1}_{\text{Term Freq}} \times (1.2 + 1)}{1 + 1.2 \times \underbrace{1}_{k_1}} \right\}$$

0.470

For learning $\rightarrow$ 0.134

$$\text{BM25}(D_1, Q) = 0.470 + 0.134 = \underline{\underline{0.6}}$$

✓ BM25(D_1, Q) .

TF
w

✓ BM25(D_2, Q)

✓ BM25(D_3, Q)

100
w

Dense Search $\rightarrow$ fast but slightly inaccurate

BM25 $\rightarrow$ slow but accurate

BM25

<u>Rank</u>	<u>Document</u>
1	D ₁
2	D ₃
3	D ₂
4	D ₅

Dense

<u>Rank</u>	<u>Document</u>
1	D ₂
2	D ₁
3	D ₄
4	D ₃

Calculate RRF for D₁

K=60

$$RRF(D_1) = \left(\frac{1}{60+1} \right) + \left(\frac{1}{60+2} \right) = \underline{\underline{0.03}}$$

$$RRF(D_2) = \underline{\underline{0.04}}$$

$$RRF(D_3) = \underline{\underline{0.03}}$$

Retrieval

→ Top 5
nm

{ symmetric
sim

+
Exact (BM25)

→ Conf Score

(UQ
nm) ~ Top 5
nm

→ Rearrangement

→ Cross Encoder
Rank
nm

UQ Vs 100,000 text
top 5

ANNT (Accuracy ↓)

↳ Top 5 ✓

1M → Top 5

↳ Cross Encoder

top-p & top-k

ways for LLM to generate output

UQ = Apple CEO

$D_1 \Rightarrow$ Apple is a fruit

$D_2 \Rightarrow$ Tim cook is CEO of apple

$D_3 \Rightarrow$ I love apple pie

UQ Vs Document

ANN

Top 3

→ Me-mark

Apple CEO $\sim D_2$ ($\uparrow$ conf)
Cross encoder.

2 Cross-Encoder Re-ranking

Better Relevance by Understanding Interactions

Cross-encoders evaluate the query and document together, capturing fine-grained interactions between words to produce more accurate relevance scores.

Why Re-rank?

Initial retrieval (BM25/Dense/Hybrid) is fast but not perfectly ordered.

Example Query:

"How can I prevent jailbreak attacks?"

Initial Retrieved Results (Unsorted)

1 Security Guidelines

2 Prompt Engineering

3 Guardrails

4 AI Ethics

 Actually, "Guardrails" should be the most relevant!

Bi-Encoder vs Cross-Encoder

Bi-Encoder (Retrieve)

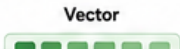
Question
How can I prevent jailbreak attacks?

Encoder
Embedding Model

Vector


Document
Guardrails are effective against jailbreaks.

Encoder
Embedding Model

Vector


• Encodes question and document **separately**. Fast, but misses fine-grained word interactions.

Cross-Encoder (Re-rank)

Question + Document
How can I prevent jailbreak attacks?
[SEP]
Guardrails are effective against jailbreaks.

Cross-Encoder
(Transformer)

Relevance Score
0.97

• Encodes question and document **together**. Captures interactions between every question word and every document word.



Goal

Reorder the top retrieved documents so the **most relevant** ones appear at the top.

How Cross-Encoder Works

The cross-encoder reads the full pair as one sequence.

[CLS] How can I prevent ... ? [SEP] Guardrails are effective ... [SEP]

Transformer Encoder



The [CLS] representation is used to predict a relevance score.

Re-ranking Effect

Before Re-ranking (from Retriever)

1 Security Guidelines

2 Prompt Engineering

3 Guardrails

4 AI Ethics

Cross-Encoder Re-ranker

After Re-ranking

1 Guardrails

2 Security Guidelines

3 Prompt Engineering

4 AI Ethics

 **More accurate ordering = better answers from the LLM!**

Why Not Rerank Everything?

Cross-encoders are computationally expensive.



If we have 10,000 documents:

- Cross-encoder must run 10,000 forward passes
- Too slow and costly

Instead, we use a two-stage approach.



1. Retrieve

Use BM25 / Dense / Hybrid to get top-k documents.

Fast

Top 20 Documents





2. Rerank

Use cross-encoder to score and rerank these top-k docs.

Slower but Accurate

Top 3 Documents





3. Return

Pass top results to LLM for generating the answer.

Best Quality

Key Takeaway

Cross-encoder re-ranking significantly improves relevance by understanding the interaction between the query and content. **Retrieve broadly, then rerank narrowly.**

Documents $\rightarrow$ Chunk $\rightarrow$ VE $\rightarrow$ VDB

UQ $\rightarrow$ Vs VDB

$\rightarrow CS_1, CS_2, CS_3 \dots$

Small Evaluation Dataset

Query Answer
└──────────┘
golden dataset

{ Recall@K
 precision@K
 NDCG ✓

3 Retriever Comparison

BM25 vs Dense vs Hybrid

Each retriever has its strengths and weaknesses. Hybrid search combines the best of both worlds for highest overall performance.



Goal

Choose the **right retriever** for your use case or combine them for **best results**.



BM25 (Lexical Search)

Matches exact keywords using statistical term importance.



Best For

- ✓ Exact keywords, IDs, numbers, codes
- ✓ Proper nouns, names, acronyms
- ✓ Error codes, invoice numbers, tickets
- ✓ Short queries with specific terms



Weaknesses

- ✗ Doesn't understand meaning or context
- ✗ Poor performance on synonyms and paraphrases
- ✗ Sensitive to wording variations

Example

Query: "INC-2847 certificate expired"

Document	BM25 Score
Doc A (INC-2847) certificate expired	3.91
Doc B SSL certificate issue	1.12
Doc C Expired cert caused outage	0.45
Doc D General security guidelines	0.10

★ **Strength:** Finds exact matches with high precision



Dense (Semantic Search)

Uses embeddings to capture meaning and context.



Best For

- ✓ Paraphrases and synonyms
- ✓ Concepts and topics
- ✓ Long natural language queries
- ✓ Semantic understanding



Weaknesses

- ✗ Struggles with exact identifiers and numbers
- ✗ May miss rare keywords
- ✗ Embedding quality depends on model

Example

Query: "How do I stop prompt attacks?"

Document	Dense Score
Doc A Prompt injection prevention guide	0.92
Doc B Jailbreak defense strategies	0.89
Doc C Guardrails for LLMs	0.85
Doc D Password reset process	0.21

★ **Strength:** Understands meaning and finds relevant content even with different wording



Hybrid (BM25 + Dense)

Combines lexical precision with semantic understanding.



Best For

- ✓ Both exact matches and semantic similarity
- ✓ All types of queries (short or long)
- ✓ Production RAG systems
- ✓ Maximum recall + precision



Weaknesses

- ✗ Slightly more complex to implement
- ✗ Requires tuning of weights (α , β)
- ✗ Higher compute than single retriever

Example

Query: "INC-2847 certificate expired"

Document	BM25 Score	Dense Score	Hybrid Score
Doc A (INC-2847) certificate expired	3.91	0.72	0.91
Doc B SSL certificate issue	1.12	0.68	0.78
Doc C Expired cert caused outage	0.45	0.81	0.74
Doc D General security guidelines	0.10	0.32	0.28

★ **Strength:** Best overall balance of precision and recall



Quick Summary

Retriever	Strength	Best For	Weakness
BM25	Exact match, high precision	IDs, codes, keywords	No semantic understanding
Dense	Semantic understanding	Meaning, paraphrases	Weak on exact identifiers
Hybrid	High recall + precision	Most real-world use cases	More complex to tune



Key Takeaway

For most RAG systems, **Hybrid Search** is the recommended choice as it leverages the strengths of both BM25 and Dense retrieval.



Retrieve broadly with Hybrid (top 20–50), then re-rank with Cross-Encoder for maximum relevance.

4 Query Expansion & Reformulation

Improve Recall by Asking Better Questions

Users often ask short or ambiguous questions. Expanding or reformulating the query helps retrieve more relevant documents.



Goal

Increase **recall** by generating multiple perspectives or **better versions** of the query.



A. Query Expansion (Add More)

Generate related terms, synonyms, and alternative phrasings to search more broadly.

Expanded Queries (Examples)

- Password reset
- Forgot password
- Credential recovery
- Authentication failure
- Unable to login

Search All Queries



Retrieve and merge results



Why it helps

Covers more ways the information might be expressed.
Increases recall by exploring multiple angles.



B. Query Reformulation (Rewrite Better)

Rewrite the original query to make it clearer, more specific, and easier to understand for the retriever.

User Query (Original)

"How do I stop prompt attacks?"

Reformulated Query (LLM)

"How can prompt injection attacks be prevented?"

Better Retrieval



Higher quality retrieval results



Why it helps

Fixes vague or poorly phrased queries.
Aligns the query with how information is written in documents.



C. Multi-Query Retrieval (Generate Many, Search All)

Generate multiple diverse queries and combine results for maximum coverage.

Generated Queries (Examples)

- Prompt injection
- Jailbreak defense
- Guardrails
- Prompt security
- Preventing LLM jailbreaks

Search Each Query



Merge + Deduplicate Results



Why it helps

Captures different intents and wordings.
Significantly improves recall, especially for ambiguous queries.



When to Use



Short or vague queries

e.g., "fix issue", "login problem"



Use Expansion + Reformulation



Domain with diverse terminology

e.g., security, healthcare, legal



Use Multi-Query Retrieval



Low retrieval results / low confidence

Not enough relevant docs found



Expand or reformulate and search again

Example

Original Query

Password problem

Expanded Queries

Password reset

Forgot password

Credential recovery

Authentication failure

...

Reformulated Query

"How can I recover access if I forgot my password?"

Multi-Query Set

Password reset

Login issue

Account recovery

...



Result



More relevant documents found
= Higher recall



Key Takeaway

Better queries lead to better retrieval.

Expand to **explore more**. Reformulate to **be clearer**. Generate multiple to **cover all possibilities**.



Combine these techniques with Hybrid Search for the best results.

A
Expansion
mmmm

Search more
things

$\sqrt{B}$ B
Reformulation
mmmm

Search smarter way

Multi-query retrieval
mm

Combination of A & B

User query

Query Reformulation (LLM)

Multi-Query generation
(3-5 different scenarios)

Hybrid Search (BM25 + Vector Search)

Merge + deduplication

Cross encoder (re-ranking)

Top_{q_m}k

LLM (QA + Top_k context)
(generation)

$Q_1 \rightarrow \text{Top } k$
 $Q_2 \rightarrow \text{Top } k$
 $Q_3 \rightarrow \text{Top } k$

k=5

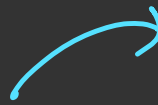
chat bot
~~~~

Frontend  
~~~~

↳ API Endpoint
Request / send



User
~~~~



AI  
Data Engg  
system  
(Backend)

# 5 Relevance Scoring & Confidence

## Know When to Answer and When to Say “I Don’t Know”

Relevance scoring helps us measure how well retrieved documents match the query. Confidence thresholds ensure we answer only when we have sufficient evidence, reducing hallucinations.



### Goal

Estimate **confidence** in retrieved context and avoid generating answers when **evidence** is weak.



### What is Relevance Score?

A numerical value that indicates how relevant a document is to the query. Commonly based on cosine similarity, BM25 score, or cross-encoder score.

#### Example: Retrieved Documents

| Rank | Document                           | Relevance Score |
|------|------------------------------------|-----------------|
| 1    | Doc A: Guardrails for LLMs         | 0.91            |
| 2    | Doc B: Prompt Injection Prevention | 0.88            |
| 3    | Doc C: Security Best Practices     | 0.82            |
| 4    | Doc D: AI Ethics Overview          | 0.45            |
| 5    | Doc E: Company Policies            | 0.28            |

#### How Scores are Computed



##### Dense Similarity

Cosine similarity between query and document embeddings.



##### BM25 Score

Lexical match based on  $TF \times IDF$ .



##### Cross-Encoder Score

Relevance score from a cross-encoder (0 to 1).



### Why Confidence Thresholds Matter

Low-relevance context often leads to hallucinations or incorrect answers.

#### Low Confidence (Weak Evidence)

Scores: 0.31, 0.28, 0.20



Risk: High chance of wrong answers



#### System response:

“I couldn’t find sufficient evidence”

#### High Confidence (Strong Evidence)

Scores: 0.91, 0.88, 0.82



Risk: Low chance of wrong answers



#### System response:

Generate accurate answer



### Confidence Threshold Guidelines

| Score Range | Confidence Level | Action                                            | Example System Response                                       |
|-------------|------------------|---------------------------------------------------|---------------------------------------------------------------|
| > 0.80      | ★★★ High         | ✅ Answer confidently                              | “Here is the answer based on the documents.”                  |
| 0.60 – 0.80 | ★★ Medium        | ⚠️ Answer cautiously (include caveats)            | “Based on the information available, it seems that...”        |
| < 0.60      | ★ Low            | ❌ Do not answer / Ask to rephrase / Retrieve more | “I couldn’t find enough relevant information to answer this.” |



### Best Practices

- ✓ Use cross-encoder scores for final confidence estimation.
- ✓ Calibrate thresholds using validation data.
- ✓ Consider average score, top-k score and score distribution.
- ✓ Combine with other signals (e.g., diversity, recency).
- ✓ Always prefer admitting uncertainty over wrong answers.

### Typical Score Distribution



### How It Fits in the RAG Pipeline



#### 1. Retrieve

Retrieve top-k documents using hybrid search.



#### 2. Score

Compute relevance scores for each document.



#### 3. Evaluate Confidence

Compare scores against thresholds.



#### High Confidence

Proceed to LLM Generation.



#### Low Confidence

Reformulate query / Retrieve again or respond with uncertainty.



#### 4. Generate / Respond

Generate answer with high-quality context or respond appropriately.



### Key Takeaway

Relevance scores + **confidence thresholds** = **reliable answers**.  
It's better to say “I don’t know” than to give a wrong answer.



TIP

Tune thresholds for your domain and user expectations.  
Monitor and improve continuously.

# 6 Self-Correcting RAG

## Let the System Check Itself and Correct Course

Self-Correcting RAG adds a verification step between retrieval and generation. The system evaluates the retrieved context and decides whether to proceed, refine the query, or retrieve more information.

### The Problem with Traditional RAG

Traditional RAG retrieves some documents and immediately generates an answer, even if the context is weak or irrelevant.

Retrieve

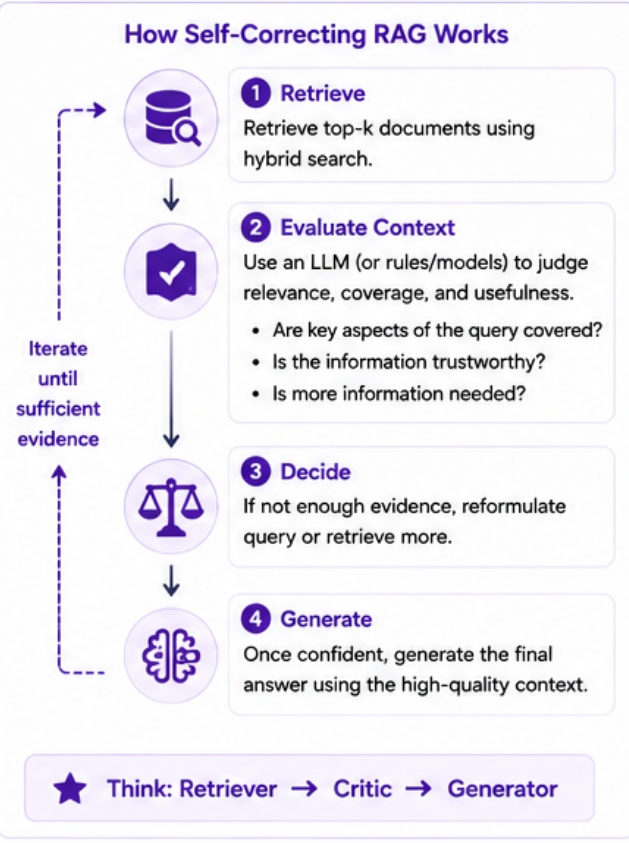
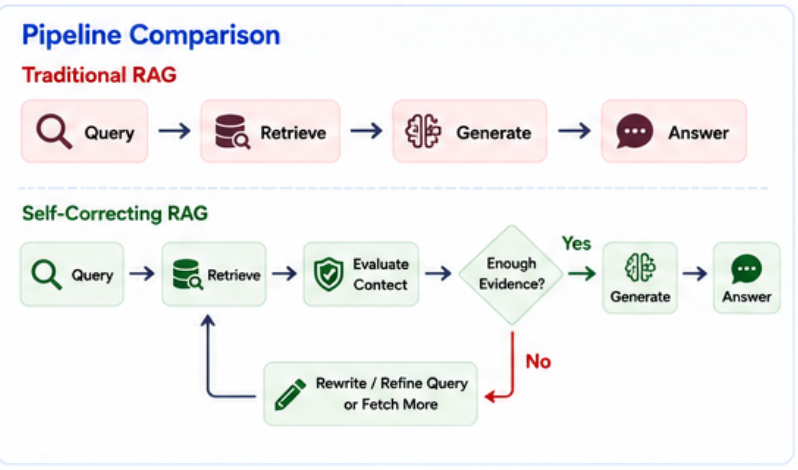
 → 

LLM Generate

 → 

Answer

Leads to wrong or hallucinated answers when evidence is insufficient.



### Goal

Reduce **hallucinations** by ensuring the model generates answers only when the retrieved context is sufficient and **relevant**.

### Benefits

**Reduces Hallucinations**

Generates answers only when there is enough reliable evidence.

**Higher Accuracy**

Iteratively improves retrieval until the context is strong.

**Better User Trust**

The system knows when to say "I don't have enough information."

### Example

Query: "How did incident INC-2847 affect customers?"

| Step           | What Happens                                                | Outcome         |
|----------------|-------------------------------------------------------------|-----------------|
| 1 Retrieve     | Initial docs are about database outage, no customer info.   | Low Confidence  |
| 2 Evaluate     | Missing key aspect: customer impact.                        | Not Enough      |
| 3 Refine Query | LLM adds: "customer impact, user facing issues, postmortem" | Retrieve Again  |
| 4 Re-evaluate  | Now docs include customer tickets and impact report.        | High Confidence |
| 5 Generate     | Answer generated using high-quality, relevant context.      | Accurate Answer |

### Best Practices

Use a strong evaluator (LLM with clear rubric) or reliable signals.

Define clear criteria: relevance, coverage, trustworthiness.

Limit iterations to avoid latency (e.g., max 2-3 loops).

Combine with hybrid search and cross-encoder re-ranking for best results.

### Signals for Evaluation

Relevance Score (similarity)

Coverage of key aspects

Source Quality / Trustworthiness

Recency & Freshness

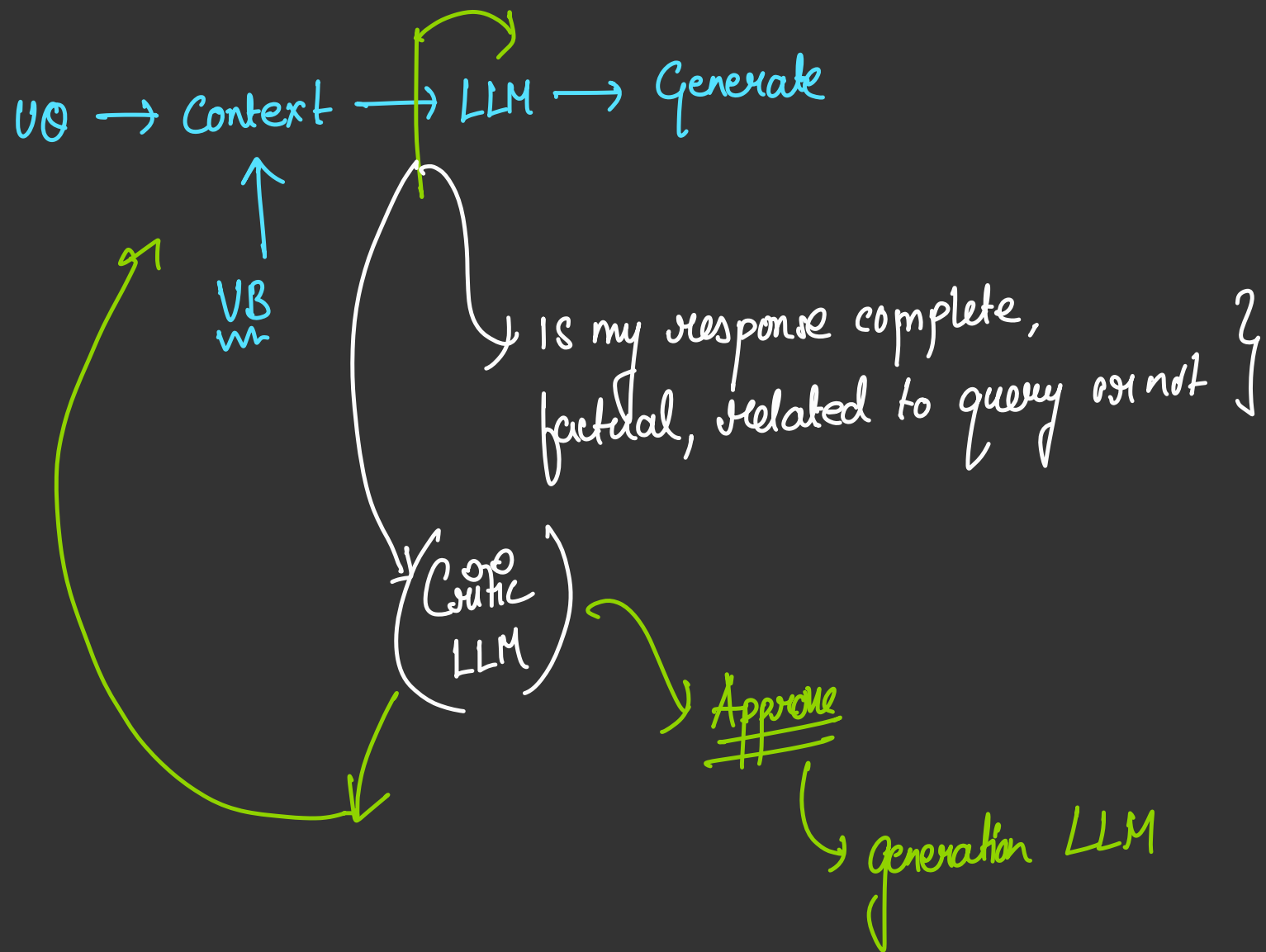
Uncertainty / Ambiguity

### Key Takeaway

Self-Correcting RAG adds a critical feedback loop to verify and improve context quality. It **retrieves smartly**, checks carefully, and **answers confidently**.

**TIP**

Better to say "I don't know yet" and search again than to give a confident but wrong answer.



①



Content Hash  
(SHA / MD5)  
256

Detect

Exact duplicates

Speed

Very fast

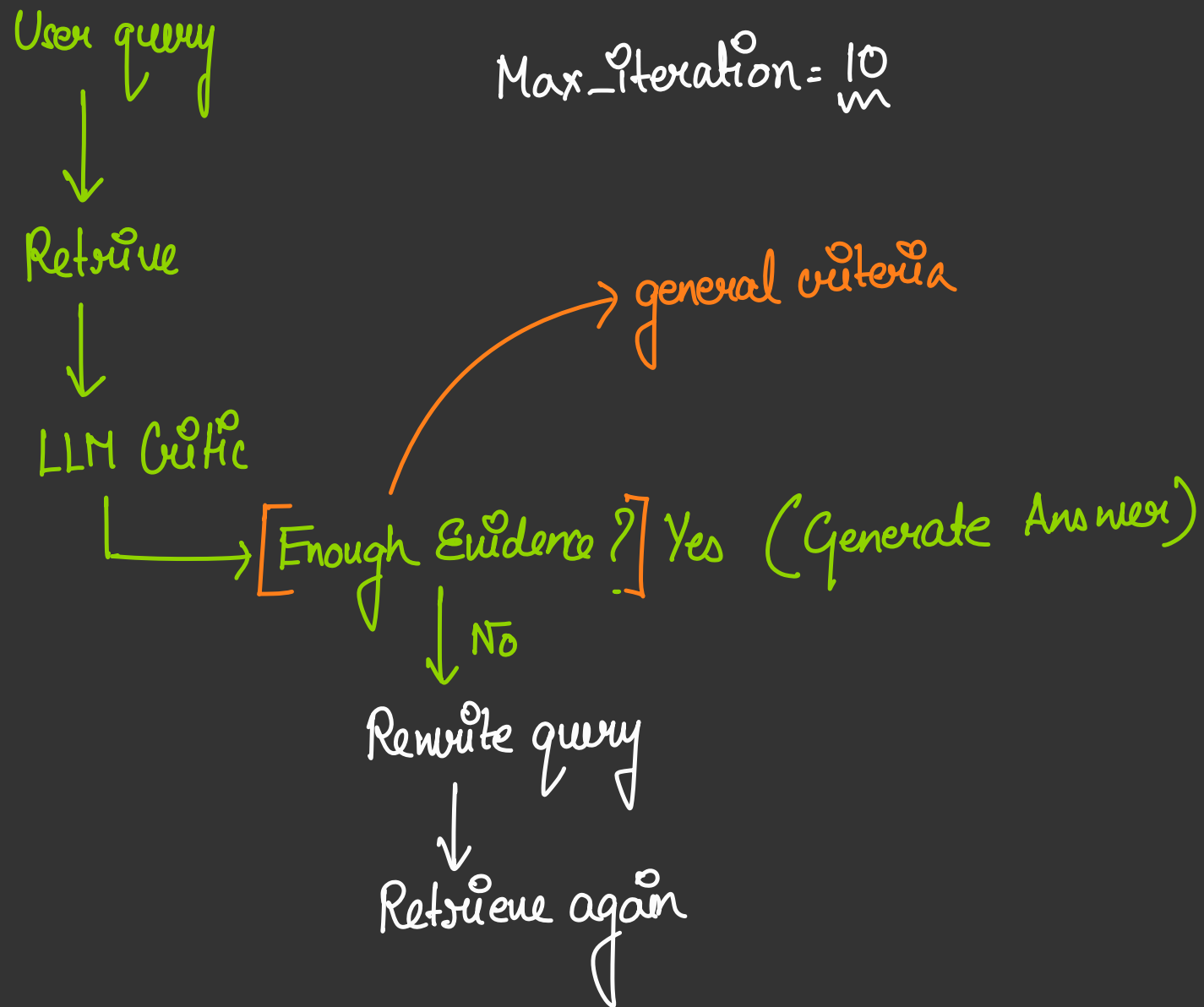
②



Mini Hash

Approx / Near  
duplicates

Medium



# 7 Iterative Retrieval

## Search, Learn, and Search Again

Iterative Retrieval improves results by retrieving, evaluating, and refining the query multiple times until enough relevant information is found.



### Goal

Find the most relevant information by refining the search step-by-step instead of relying on a single retrieval attempt.



### Why Iterative Retrieval?

- Initial queries may be too narrow or unclear.
- Retrieving once can miss important context.
- Refining the query using feedback improves recall and answer quality.
- Common in complex questions and large corpora.



### When It Helps

- Complex, multi-faceted questions
- Vague or underspecified user queries
- Low-confidence or insufficient results
- Enterprise search across large knowledge bases

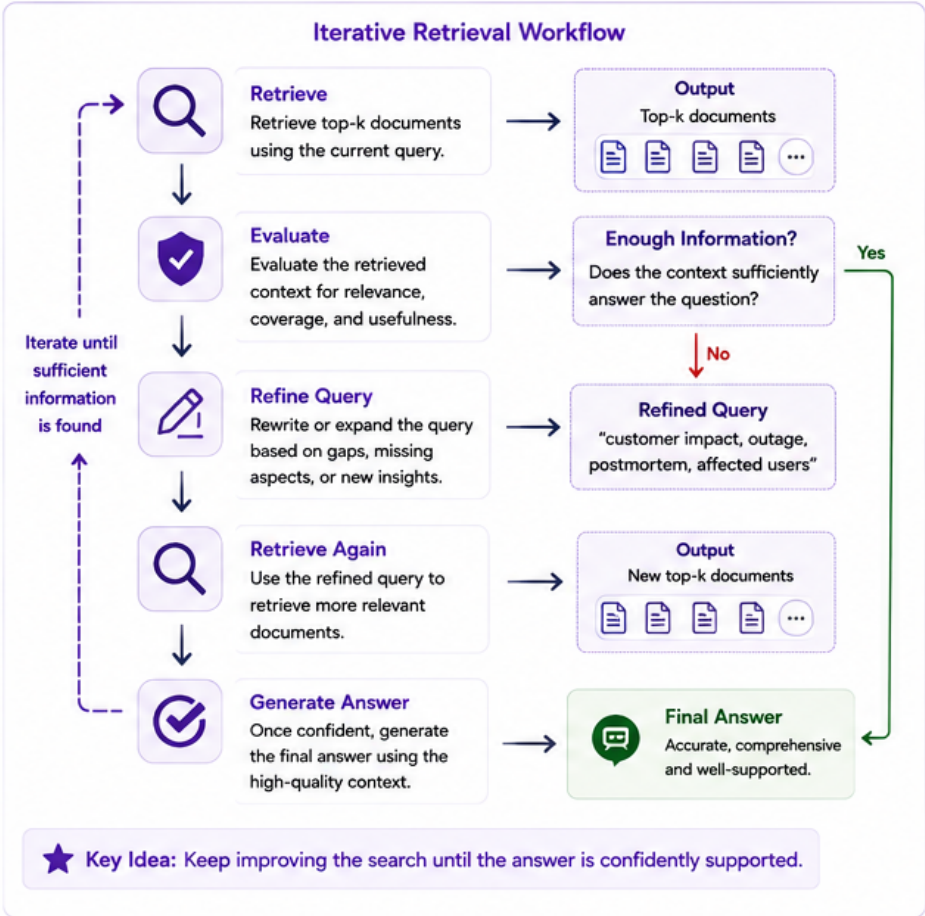


### Example Scenario

User Question:  
"How did incident INC-2847 affect customers?"

Initial query may retrieve docs about database outage but miss customer impact details.

Iterative retrieval expands the query, explores related concepts, and finally finds the postmortem with customer impact.



### Example: Iterative Retrieval in Action

Question: "How did incident INC-2847 affect customers?"

| Iteration | Query                         | What We Find                                              | Evaluation                        | Decision      |
|-----------|-------------------------------|-----------------------------------------------------------|-----------------------------------|---------------|
| 1         | INC-2847                      | Docs about database outage, certificate expiry            | Partial info. No customer impact. | Refine        |
| 2         | INC-2847 customer impact      | Some mentions of service disruption, but limited details  | Still not enough detail.          | Refine        |
| 3         | INC-2847 postmortem customers | Postmortem doc with affected customers, downtime, refunds | Sufficient and relevant.          | Stop & Answer |

### Benefits

- Higher Recall**  
Discovers information that might be missed in a single retrieval.
- Better Accuracy**  
Answers are based on more complete and relevant context.
- Adaptable**  
Works well for diverse queries and dynamic knowledge bases.
- Robust**  
Handles ambiguity, complexity, and evolving information.

### Best Practices

-  Set a maximum number of iterations (e.g., 3-5) to control latency.
-  Use clear evaluation criteria: relevance, coverage, usefulness.
-  Refine queries using gaps, synonyms, and related concepts.
-  Combine with hybrid retrieval and re-ranking for best results.
-  Track confidence scores to decide when to stop.

### Signals to Decide "Enough Information"

-  High relevance score
-  Good coverage of subtopics
-  Consistent across multiple docs
-  Confidence above threshold
-  Answerable with available context



### Key Takeaway

Iterative Retrieval mimics a smart researcher—**search, evaluate, learn, and search again**. It significantly improves the quality, completeness, and reliability of RAG systems.



### TIP

Start simple: 2-3 iterations, strong evaluation, and good query refinement can dramatically improve results.

# Complete Production RAG Pipeline

## From User Question to Accurate, Reliable Answer

A modern RAG system combines multiple techniques to maximize relevance, minimize hallucinations, and deliver high-quality answers.



### Goal

Retrieve the most relevant context, validate it, and generate accurate answers with high confidence.

### What Each Step Solves

- 1 Captures the user's intent.
- 2 Improves recall by exploring different ways to ask the question.
- 3 Combines strengths of keyword search (exact match) and semantic search (meaning).
- 4 Finds the most relevant documents by deeply understanding query-doc interactions.
- 5 Prevents wrong or hallucinated answers by checking if we have enough evidence.
- 6 Fixes weak retrieval by learning from gaps and searching again.
- 7 Generates accurate, grounded, and well-structured answers.
- 8 Provides transparency with sources for trust and verification.

### Key Benefits

- Higher Accuracy**  
Better retrieval + re-ranking + validation.
- Lower Hallucinations**  
Confidence check & iterative refinement.
- Better Recall**  
Hybrid search + query expansion.
- User Trust**  
Citations and reliable answers.

### Best Practices



**Keep Index Fresh**  
Update embeddings and BM25 index regularly.



**Tune Thresholds**  
Calibrate confidence thresholds using real user data.



**Monitor & Evaluate**  
Track retrieval metrics (recall@k, hit rate, answer quality).



**Optimize Latency**  
Re-rank only top-k documents, cache embeddings.



**Ensure Safety**  
Add guardrails, PII filters, and content policies.



### Key Takeaway

A high-quality RAG system is not just about retrieval or generation—it's about the right combination of techniques in the right order.



Retrieve smart.  
Validate carefully.  
Generate confidently.

Hybrid Search  $\rightarrow$  Top 20

$\downarrow$   
Cross Encoder (Re-ranking)

|                    |          |          |
|--------------------|----------|----------|
| <del>✓</del> $D_1$ | $V_S$    | $V_Q$    |
| $D_2$              | $V_S$    | $V_Q$    |
| <del>✓</del> $D_3$ | $V_S$    | $V_Q$    |
| $D_4$              | .        | $V_Q$    |
| $\vdots$           | $\vdots$ | $\vdots$ |
| $D_{20}$           | .        | .        |

$\rightarrow$  Top 3  
 $D_1, D_3$   
 $\swarrow \searrow$   
0.60      0.53

Excess Info



Easy to Hallucinate